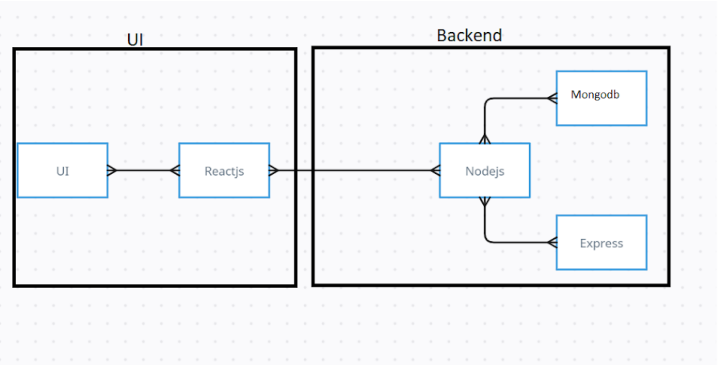
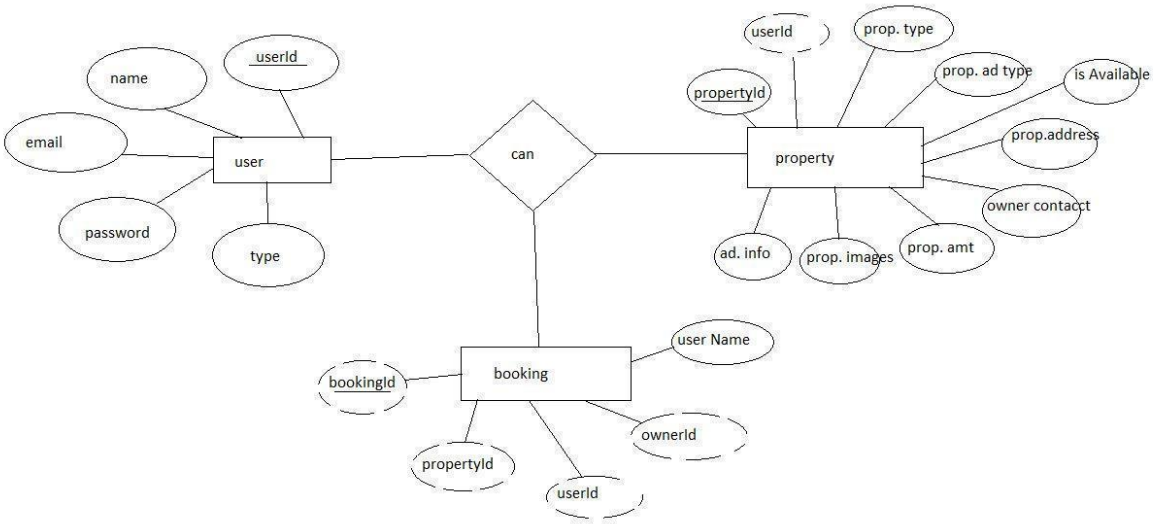
PROJECT ARCHITECTURE



The technical architecture of our House rent app follows a client-server model, where the frontend serves as the client and the backend acts as the server. The frontend encompasses not only the user interface and presentation but also incorporates the axios library to connect with backend easily by using RESTful Apis.

The frontend utilizes the bootstrap and material UI library to establish real-time and better UI experience for any user whether it is admin, doctor and ordinary user working on it.

On the backend side, we employ Express.js frameworks to handle the server-side logic and communication.
For data storage and retrieval, our backend relies on MongoDB. MongoDB allows for efficient and scalable storage of user data, including user profiles, for booking room, and adding room, etc. It ensures reliable and quick access to the necessary information



Entities & Attributes

User

Attributes:
UserID (PK), Name, Email, Phone, Password, UserType (Tenant / Landlord / Agent), ProfileImage, CurrentLocation

Explanation:
Represents all users of the app. UserType helps distinguish between renters and property owners.
Property
Attributes:
PropertyID (PK), OwnerID (FK → User), Title, Description, Location, RentAmount, PropertyType, FurnishingStatus, Amenities, Images, Status (Available/Booked)

Explanation:
Contains all details of a listed property. Linked to the User who owns it.
3) **Booking**
Attributes:
BookingID (PK), PropertyID (FK), TenantID (FK → User), StartDate, EndDate, BookingDate, Status (Confirmed/Pending/Cancelled)

Explanation:
Represents the booking made by a tenant for a property.

4) BookingHistory
Attributes:
HistoryID (PK), UserID (FK), PropertyID (FK), BookingID (FK), ViewedOn

Explanation:
Keeps a record of all previous bookings and views for users

Relationships Overview
User ↔ Property:
One user (landlord) can own many properties (1:N).

User ↔ Booking:
A user (tenant) can make multiple bookings (1:N).

Booking ↔ Property:
One booking is made for one property (N:1).

User ↔ SupportTicket:
Users can raise multiple tickets (1:N).

Description

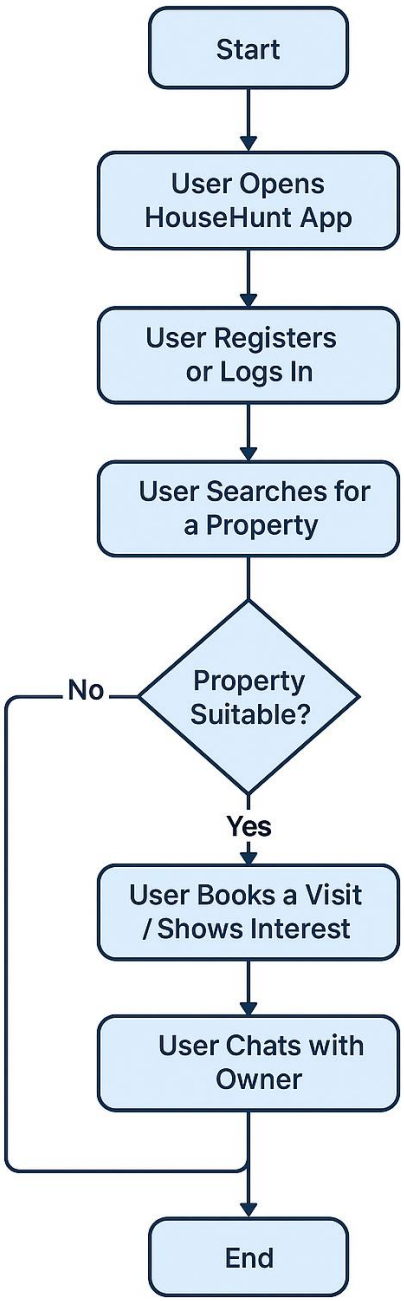
1. Advanced Property Search
Filters: Users can filter properties based on location, price range, property type, size, number of bedrooms, and specific amenities (e.g., pet-friendly, pool, garage).
Saved Searches: Allows users to save their search criteria for easy access to new listings that match their preferences.

2. Market Insights and Trends

Price Trends: Displays information on how property prices are trending in specific areas, allowing buyers and sellers to make informed decisions.

Neighbourhood Insights: Information on local amenities, schools, crime rates, and transport options, giving users a deeper understanding of the area they are considering.

Investment Potential: Provides data on property value appreciation and investment opportunities for users looking to invest in real estate.



User Flow

This updated user journey reflects only the features that are **actually implemented** in your current HouseHunt application.

1. Start

The user begins their journey to find a rental property.

2. User Opens HouseHunt App

The user launches the HouseHunt app on their smartphone or web browser.

3. User Registers or Logs In

New users can register with basic information.

Existing users can log in using saved credentials.

4. User Searches for a Property

Using search filters like location, rent range, property type, and amenities, the user browses available listings.

5. Property Suitable

If the user finds a suitable property, they proceed to contact the owner.

If not, they continue exploring other listings.

6. User Chats with Owner

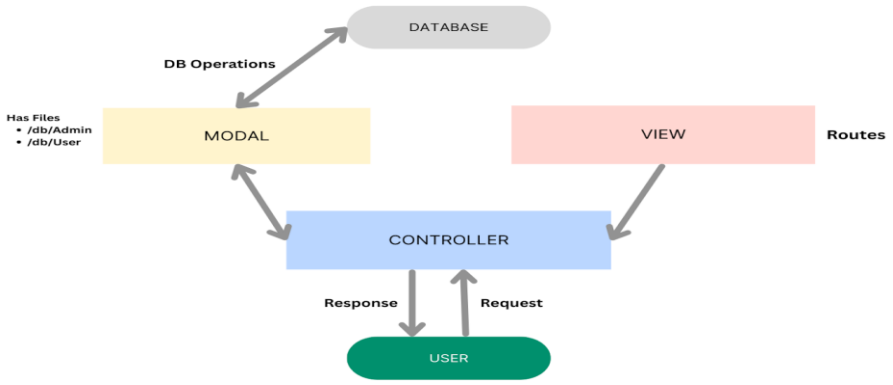
The app enables the user to directly message or contact the property owner for further discussion.

7. End

Once the user finalizes the deal outside the app, their in-app journey ends.

All future communication and formalities (like agreements and payment) happen offline

MVC PATTERN:



The HouseRent application follows the **Model-View-Controller (MVC)** architectural pattern, a software design approach that separates an application into three interconnected layers. This separation allows for modularity, easier maintenance, and scalability.

Model Layer (Data Layer)

The **Model** layer is responsible for handling all data-related logic. This includes the definition of data schemas and the operations performed on the database using those schemas. The models are implemented using **Mongoose**, which provides a schema-based solution to model application data for MongoDB.

Controller Layer

The **Controller** layer acts as an intermediary between the view (routes) and the model. It receives incoming requests, processes the input (which may include validation or transformation), calls the appropriate methods from the model, and then returns a response to the client.

View Layer (Routing Layer)

In the context of a backend REST API, the **View** is implemented as the **routing layer**, where various endpoints are defined. These endpoints determine how the backend responds to different HTTP requests (GET, POST, PUT, DELETE) and are responsible for invoking the appropriate controller functions.

Advantages of Using MVC in This Project

Separation of Concerns: Each layer has a clearly defined responsibility, improving readability and maintainability.

Scalability: New features can be added easily by creating new routes, controllers, and models.

Reusability: Logic in controllers and models can be reused across multiple parts of the application.

Testing: Each layer can be tested independently, especially the controllers and models.
Collaboration-Friendly: Multiple developers can work simultaneously on different layers without conflict.